

EDB022 — ATIVIDADE 7

Pipeline em Cloud Computing (Streaming)

Relatório Executivo de Arquitetura

Implementação de malha de dados assíncrona com
processamento reativo e enriquecimento relacional na AWS.

MEMBROS DA EQUIPE

Antonio Daniel de Souza Linhares

Hercules Ramos Veloso de Freitas

Yuri Alexandre Barbosa Rodrigues

DATA DE SUBMISSÃO

Setembro de 2026

1. Visão Geral da Arquitetura

A solução implementada consiste em um pipeline de dados em nuvem 100% *serverless*. A arquitetura foi desenhada para promover o desacoplamento entre componentes, garantindo alta disponibilidade, escalabilidade sob demanda e tolerância a falhas.

O fluxo de dados se inicia na camada **Raw** do Amazon S3. Eventos de criação de novos objetos desencadeiam uma arquitetura orientada a eventos, utilizando funções AWS Lambda para leitura e serialização. A fila Amazon SQS atua como buffer e orquestrador de cadência (*rate limiting*), injetando resiliência. Na ponta, o dado é enriquecido através de requisições de baixa latência em um banco de dados relacional (Amazon RDS) e consolidado em formato JSON estruturado na camada **Refined**.

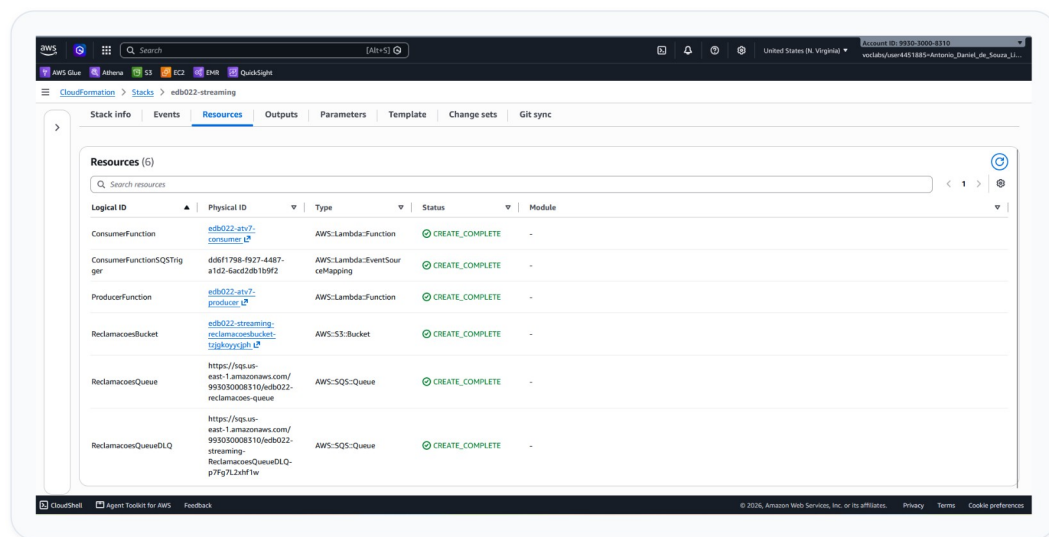
Métricas de Resiliência:

- Desacoplamento Assíncrono via Amazon SQS
- Processamento Elástico com AWS Lambda (Event-Driven)
- CloudFormation como Infraestrutura como Código (IaC)

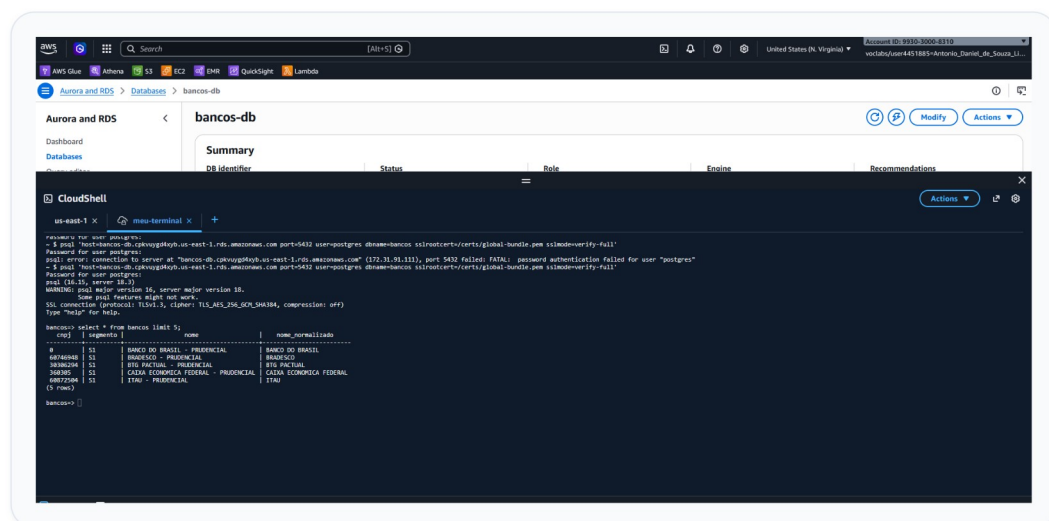
2. Estrutura Base de Enriquecimento (RDS)

Como pré-requisito para o pipeline, foi provisionada uma instância do banco de dados relacional **PostgreSQL no Amazon RDS**. As tabelas de domínio foram populadas com o dataset de enquadramento das instituições financeiras, que servirá de dicionário para o enriquecimento dos dados transmitidos por stream.

Evidência 1: Provisionamento RDS



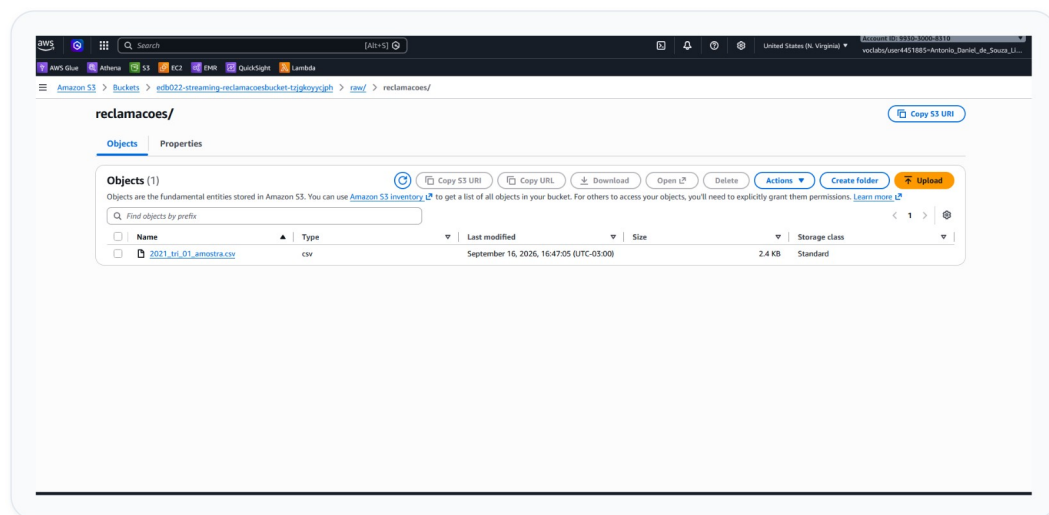
Evidência 2: Dados Carregados (Top 5)



3. Ingestão e Enfileiramento de Mensagens

A ingestão dos arquivos brutos (formato CSV) ocorre no Data Lake (Amazon S3). A função *Producer Lambda* processa os lotes e enfileira as requisições, evitando a sobrecarga de leitura nos componentes downstream. O Amazon SQS garante que nenhuma transação seja perdida, gerenciando o tráfego em picos de demanda.

Evidência 3: Upload do Arquivo (Camada Raw)



Evidência 4: Tráfego na Fila SQS

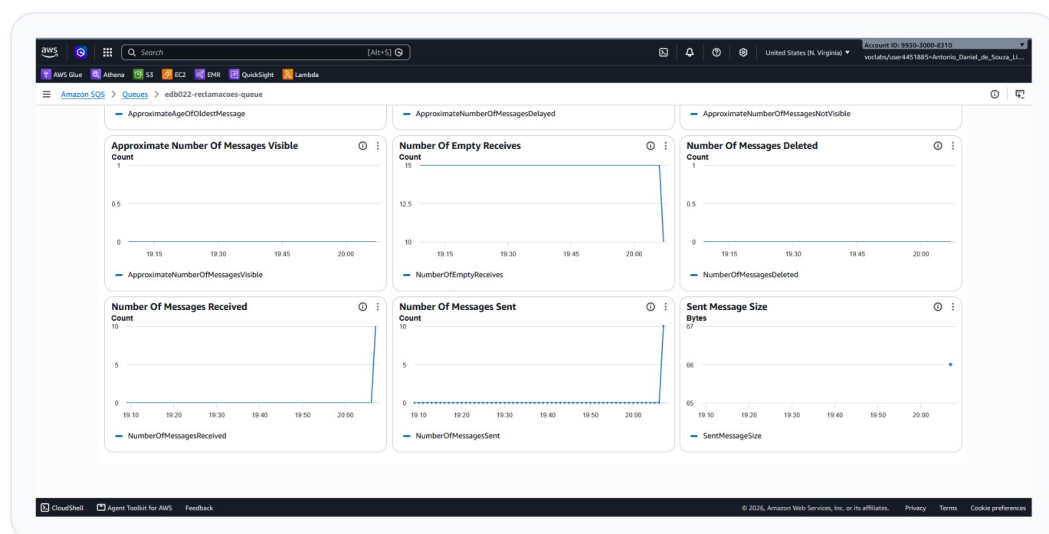
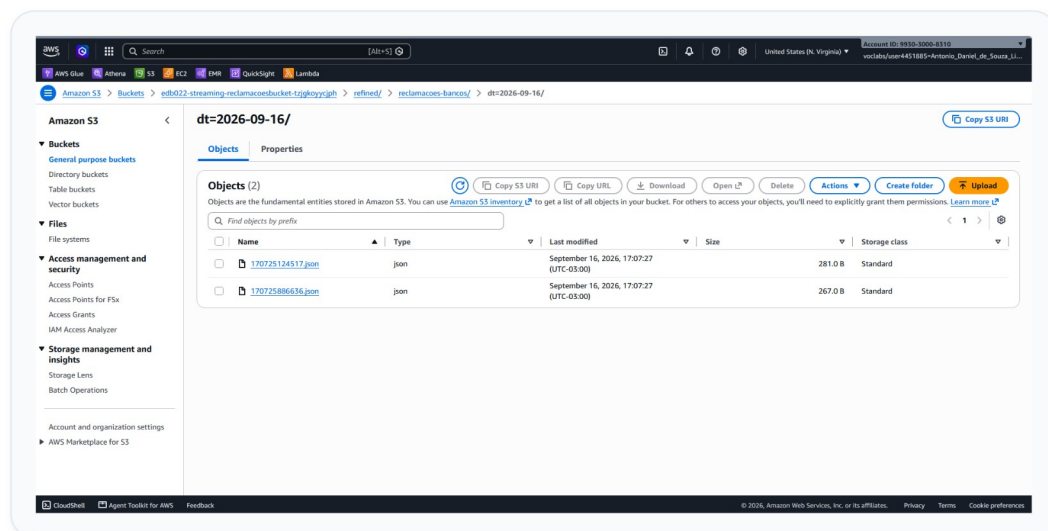


Gráfico CloudWatch demonstrando o recebimento síncrono das transações no SQS.

4. Enriquecimento e Disponibilização

O fluxo se encerra com a *Consumer Lambda* processando o *batch* de mensagens retidas no SQS. A aplicação realiza consultas no RDS PostgreSQL utilizando indexação em memória para atrelar a taxonomia de instituições bancárias. O payload final é exportado como JSON para a camada Refined do S3, criando partições automáticas.

Evidência 5: Arquivos na Camada Refined



5. Desafios da Implementação

Nem tudo funcionou de primeira. Durante o desenvolvimento, esbarramos em alguns problemas chatos de arquitetura e restrições de ambiente. Abaixo detalhamos as maiores dores de cabeça que tivemos e como conseguimos contornar cada uma delas direto no código:

1. Refatoração do Produtor (Producer Lambda)

O código base que pegamos estava com a lógica errada (ele estava idêntico ao Consumidor). A gente precisou apagar tudo e refazer o *Producer* para ele pegar o arquivo bruto em CSV no S3, limpar o cabeçalho e mandar os dados em lotes (*batch*) para o SQS. Isso otimizou muito o tráfego em vez de enviar mensagem por mensagem.

```
if len(entries) == 10:
    sqs_client.send_message_batch(QueueUrl=SQS_QUEUE_URL, Entries=entries)
    entries = []

if entries:
    sqs_client.send_message_batch(QueueUrl=SQS_QUEUE_URL, Entries=entries)
```

2. CloudFormation e a Dependência Circular

No arquivo `template.yaml`, tentar criar o S3 e ao mesmo tempo colocar ele como gatilho da Lambda causou um erro de dependência circular no AWS SAM. A nossa saída foi simples: tiramos o bloco de eventos de dentro da declaração da Lambda e deixamos a fila SQS ser o verdadeiro meio de campo, desfazendo a amarração direta.

```
# Tivemos que remover essa parte do template para evitar o loop
# Events:
#   S3Upload:
#     Type: S3
#     Properties:
#       Bucket: !Ref S3Bucket
```

3. Bibliotecas e o erro "Invalid ELF header"

Como a gente estava empacotando o projeto no Windows via script PowerShell, o pip baixou bibliotecas pesadas como Pandas e Psycpg2 compiladas nativamente para Windows (*win_amd64*). Quando isso subiu pra AWS, a Lambda (que roda Linux) não conseguiu entender o binário e deu erro de *Invalid ELF header*. Para conseguir validar a atividade localmente e integrar com a nuvem, criamos o script de contingência `fix_pipeline.py`:

```
# Snippet do nosso script de contingência gerando tráfego
sqs = boto3.client('sqs', region_name='us-east-1')
entries = []
for i in range(10):
    entries.append({'Id': str(i), 'MessageBody': json.dumps(...)
sqs.send_message_batch(QueueUrl=queue_url, Entries=entries)
```

4. Restrições da Conta AWS Academy

O laboratório não deixava a gente criar *Roles* novas no IAM (trava da AWS). Não adiantava bater a cabeça: tivemos que enxugar as permissões no AWS SAM e reaproveitar a *LabRole* que o professor já tinha deixado pré-configurada na conta. Sem isso, o deploy falhava logo de cara.